

audit-linux (adapté pour tec.tech)

Ceci est la version de l'audit Linux jadis développé par Bernard Maison, qui fonctionnait avec le BOLC et qui a été adapté pour permettre l'envoi des données d'audit vers tec.tech au lieu du BOLC.

Pour mémoire, il se base sur la [dernière version publiée par Bernard](#), la 2.7.1

Fonctions et limitations actuelles

- l'outil permet, soit de créer un nouveau matériel, soit de modifier un matériel existant, le tout à partir des données d'audit générées par le code *historique* de Bernard
- son unique vocation étant de faire de l'audit, l'outil ne permet pas de modifier certains attributs déjà déclarés dans la base tec.tech ('id', 'idLot', 'idStock', 'idGroupe', 'reconditionneur', 'createdAt', 'updatedAt')
- pour l'outil, l'identifiant utilisé pour désigner un matériel est toujours son "ID ESN" (idEsn dans l'API tec.tech) conforme à la notation utilisée par l'outil dans le contexte BOLC. Par exemple: GRPC26-0043
- l'outil a été testé mais est encore jeune; il est fortement recommandé de lire les [notes d'évolution](#) et de se familiariser avec l'outil sur la base tec.tech de test (voir ci-dessous)
- certaines des données qui allaient dans le BOLC n'ont pas de point de chute naturel dans tec.tech. L'outil les regroupe dans le champ "commentaire" de tec.tech

Obtenir l'outil

- l'outil peut être téléchargé depuis le site audits.emmaus-connect.org sous la forme d'un fichier linux-x.y.z.tgz
- le fichier se décompresse avec une commande qui créera localement un répertoire audit-linux-x.y.z contenant l'outil:

```
tar xzf linux-x.y.z.tgz
```

Rapporter des anomalies, faire des suggestions, ...

Pour des questions générales ou des suggestions, je suggère d'utiliser [l'espace GoogleChat dédié à cet outil](#).

Si vous constatez une anomalie, vous pouvez, soit m'envoyer un mail, soit utiliser [l'espace GoogleChat](#). Merci de bien vouloir joindre à votre signalement le fichier de trace créé à chaque exécution depuis la 4.3.0 . Le nom complet de ce fichier est affiché dans les toutes premières lignes de la console; par exemple:

```
[INFO]: Nom du fichier de trace: /tmp/audit-linux-journal-20260406.112818.txt
```

Utiliser l'outil

Première utilisation

- se positionner dans le bon répertoire

```
cd audit-linux.x.y.z
```

- y créer un fichier nommé `tectech-credentials.json` avec votre éditeur favori et y placer les données suivantes:

```
{
  "client_id": " *valeur à demander à l'équipe tec.tech* ",
  "client_secret": " *valeur à demander à l'équipe tec.tech* "
}
```

en respectant scrupuleusement la syntaxe (notamment les guillemets et la virgule en fin de la ligne

```
client_id )
```

Les valeurs de `client_id` et de `client_secret` sont propres à chaque ESN et doivent être obtenues auprès de l'équipe tec.tech.

Utilisations ultérieures

L'outil s'appelle directement avec le script `audit.sh`. Le script `menu.sh` ne doit pas être utilisé pour faire de l'audit.

Il y a ensuite plusieurs façons lancer l'audit, selon qu'on veut fournir l'*idEsn* sur la ligne de commande et selon qu'on choisit d'accéder à la base tec.tech de test (<https://tec-tech.osc-fr1.scalingo.io>) ou à celle de prod (<https://tec-tech-prod.osc-fr1.scalingo.io>).

Les différentes façons d'appeler l'outil sont (on prend l'*idEsn* `GRPC26-0043` comme exemple):

```
sudo bash audit.sh                # [1] idEsn sera demandé, la base de PROD est choisie
sudo bash audit.sh GRPC26-0043    # [2] la base de PROD est choisie
sudo bash audit.sh [PROD|TEST]    # [3] idEsn sera demandé et la base est explicitement
désignée
sudo bash audit.sh GRPC26-0043 [PROD|TEST] # [4] idEsn et la base sont explicitement désignés
```

Lors de l'audit, si l'équipement existe déjà dans tec.tech, l'outil le met à jour, sinon, il le crée, le tout avec les données collectées lors de l'audit.

Pour mémoire, les idEsn doivent se conformer au modèle:

```
^(BX|CR|GR|LI|LV|LY|MA|MB|RO|SD|ST|VI)(PC|TA)(\d{2})-(\d{4})$
```

Modification d'un matériel existant v/s création d'un nouveau matériel

L'outil d'audit ne sait faire des recherches qu'avec l'identifiant `eePCaa-nnnn` qu'on lui donne en paramètre (`GRPC26-0043` est pris ici comme exemple) ou bien le numéro de série du matériel, tel que directement extrait par l'audit.

De ma compréhension actuelle de tec.tech, pour un PC:

- reconditionné par un pro: `idMaterielReconditionneur` contient l'identifiant attribué par le pro,

- reconditionné par l'ESN: `idMaterielReconditionneur` contient l'identifiant *eePCaa-nnnn*, attribué par l'ESN.

Dans tous les cas, pour tec.tech, `idEsn` est un champ libre, que nous (bénévoles reconditionneurs) utilisons comme identifiant unique. Autrement dit, pour nous, `idEsn` est une valeur conforme à la syntaxe *eePCaa-nnnn* et unique dans tec.tech, alors que pour tec.tech, **ce champ n'est nullement contraint**.

Dans notre exemple, l'outil d'audit cherche donc le PC d'abord par le `idEsn=GRPC26-0043` puis, en cas d'échec, par `idMaterielReconditionneur=GRPC26-0043`.

- s'il trouve, tout va bien: il s'agit de la modification d'un équipement déjà déclaré dans tec.tech,
- s'il ne trouve pas:
 - si on a donné un numéro de lot, il considère que c'est une création et il la réalise,
 - sinon, il tente une recherche sur `numeroSerie`
 - s'il trouve, il s'agit d'une modification: on met à jour `idEsn` et les données d'audit,
 - sinon, il ne fait rien

Ça marche très bien si le référent déclare les PC par leur `idEsn` avant de les mettre entre les mains des bénévoles pour l'audit (procédure appliquée notamment à Grenoble).

Ça marche aussi pour des PC reconditionnés par un pro, qui ont été déclarés dans tec.tech avec un `idMaterielReconditionneur` (au format propre au pro) et un `numeroSerie`, et pour lesquels `idEsn` n'a pas encore été renseigné.

Ça marche assez bien si le référent déclare au moins le lot. Comme vu ci-dessus, l'outil d'audit sait créer des matériels dans tec.tech à partir des deux infos: `idLot` et `idEsn`.

De façon générale, à ma connaissance, la notion de doublon n'est pas définie formellement dans tec.tech au delà de l'unicité des différents *ID* (qui est requise par le SGBD sous-jacent). Par exemple, à l'heure où j'écris, il existe dans tec.tech plusieurs dizaines de cas où des matériels différents (*ID* différents) ont les mêmes `idMaterielReconditionneur+numeroSerie` ...

Notes importantes

- les paramètres sont convertis en majuscules avant exploitation
- les gestionnaires de tec.tech demandent à ce qu'on économise les jetons d'accès à la base; pour cela, l'outil stocke le jeton qu'il a acquis dans un fichier local `token-test.json` (ou `token-prod.json`) puis l'utilise tant qu'il est valide; il est donc important que l'outil puisse écrire dans le répertoire où se trouve [audit.sh](#)